NumPy Tutorial 8: Random Numbers — Generation and Real Use Cases

This chapter covers random number generation in NumPy. After completing this tutorial, students will understand how to generate random numbers, set seeds for reproducibility, and use random functions in real Data Science scenarios.

---

1. Introduction

In Data Science and Machine Learning, random numbers are used everywhere. We use them for:

- Creating test datasets for practice.
- Shuffling data before training a model.
- Simulating real world scenarios.
- Initializing weights in neural networks.

NumPy provides a powerful **random module** that makes working with random numbers very easy.

---

2. Generating Random Integers

```
import numpy as np

random_int = np.random.randint(1, 100)
print(random_int)
```

This generates one random integer between 1 and 99.

Generating an array of random integers:

```
arr = np.random.randint(1, 100, size=10)
print(arr)
```

Output (example — values will be different each time):

```
[45 82 13 67 29 91 54 36 78 12]
```

Generating a 2D array of random integers:

```
arr = np.random.randint(1, 100, size=(3, 4))
print(arr)
```

Output:

```
[[23 87 45 12]
 [67 34 90 56]
 [11 78 43 29]]
```

---

3. **Generating Random Floats**

np.random.rand — uniform distribution between 0 and 1

```
arr = np.random.rand(5)
print(arr)
```

Output:

```
[0.37 0.95 0.73 0.60 0.16]
```

2D array of random floats:

```
arr = np.random.rand(3, 3)
print(arr)
```

---

np.random.uniform — random floats between any range

```
arr = np.random.uniform(10, 50, size=5)
print(arr)
```

Output:

```
[23.5 41.2 15.8 38.9 27.4]
```

---

4. **Normal Distribution — Most Important in Data Science**

Normal distribution (also called Gaussian distribution) is the most common distribution in real world data. Test marks, heights, weights — all follow normal distribution.

np.random.randn — standard normal distribution (mean=0, std=1)

```
arr = np.random.randn(5)
print(arr)
```

Output:

```
[ 0.47 -1.23  0.89  0.32 -0.56]
```

np.random.normal — custom mean and standard deviation

```
arr = np.random.normal(loc=70, scale=10, size=1000)
print("Mean:", np.mean(arr))
print("Std:", np.std(arr))
```

Output:

```
Mean: 70.03
Std: 9.97
```

- loc = mean (center of the distribution)
- scale = standard deviation (spread of the distribution)
- size = number of values to generate

This is very useful for simulating student marks, temperatures, or any real world data.

---

5. **What is a Random Seed**

Every time we run random functions, we get different values. But in Data Science, we often need the same random values every time — for reproducibility.

We use **np.random.seed()** to fix the random values.

```
np.random.seed(42)
arr = np.random.randint(1, 100, size=5)
print(arr)

np.random.seed(42)
arr = np.random.randint(1, 100, size=5)
print(arr)
```

Output:

```
[52 93 15 72 61]
[52 93 15 72 61]
```

Both times we get the same values because we used the same seed (42). The number 42 is just a common convention — you can use any number.

---

6. **Shuffling an Array**

Shuffling means randomly rearranging the elements of an array.

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

np.random.shuffle(arr)
print(arr)
```

Output (example):

```
[ 7  3  9  1  5 10  4  8  2  6]
```

Note: shuffle changes the original array directly.

This is commonly used to shuffle training data before feeding it to a Machine Learning model.

---

7. **Random Choice — Selecting Random Elements**

```
arr = np.array([10, 20, 30, 40, 50])

single = np.random.choice(arr)
print("Single random choice:", single)
```

```
multiple = np.random.choice(arr, size=3, replace=False)
print("Multiple without repeat:", multiple)
```

Output:

```
Single random choice: 30
Multiple without repeat: [50 10 40]
```

- replace=False means no element will be selected twice.
- replace=True (default) means the same element can be selected multiple times.

---

8. **Practical Example — Simulating Student Marks**

Let us simulate a class of 50 students with normally distributed marks.

```
import numpy as np

np.random.seed(42)

student_marks = np.random.normal(loc=65, scale=15, size=50)
student_marks = np.clip(student_marks, 0, 100)
student_marks = np.round(student_marks).astype(int)

print("Sample marks:", student_marks[:10])
print("Class average:", np.mean(student_marks))
print("Highest mark:", np.max(student_marks))
print("Lowest mark:", np.min(student_marks))
print("Students who passed:", np.sum(student_marks >= 50))
```

Output:

```
Sample marks: [75 84 54 78 69 61 88 65 57 72]
Class average: 65.2
Highest mark: 98
Lowest mark: 23
Students who passed: 38
```

This is a realistic simulation of a real class — very useful for practice and testing.

---

9. **Common Random Functions Summary**

Function	Description
np.random.randint(low, high, size)	Random integers
np.random.rand(size)	Random floats 0 to 1
np.random.uniform(low, high, size)	Random floats in range
np.random.randn(size)	Standard normal distribution
np.random.normal(mean, std, size)	Custom normal distribution
np.random.seed(n)	Fix random values
np.random.shuffle(arr)	Shuffle array in place
np.random.choice(arr, size)	Pick random elements

10. **Conclusion of Tutorial 8**

In this tutorial, students have learned:

1. How to generate random integers using randint.
2. How to generate random floats using rand and uniform.
3. What normal distribution is and how to generate it.
4. What a random seed is and why it is important.
5. How to shuffle an array.
6. How to pick random elements using random choice.
7. A practical example of simulating student marks.

In the next tutorial, we will learn how NumPy and Pandas work together for powerful data analysis.

This completes NumPy Tutorial 8.